

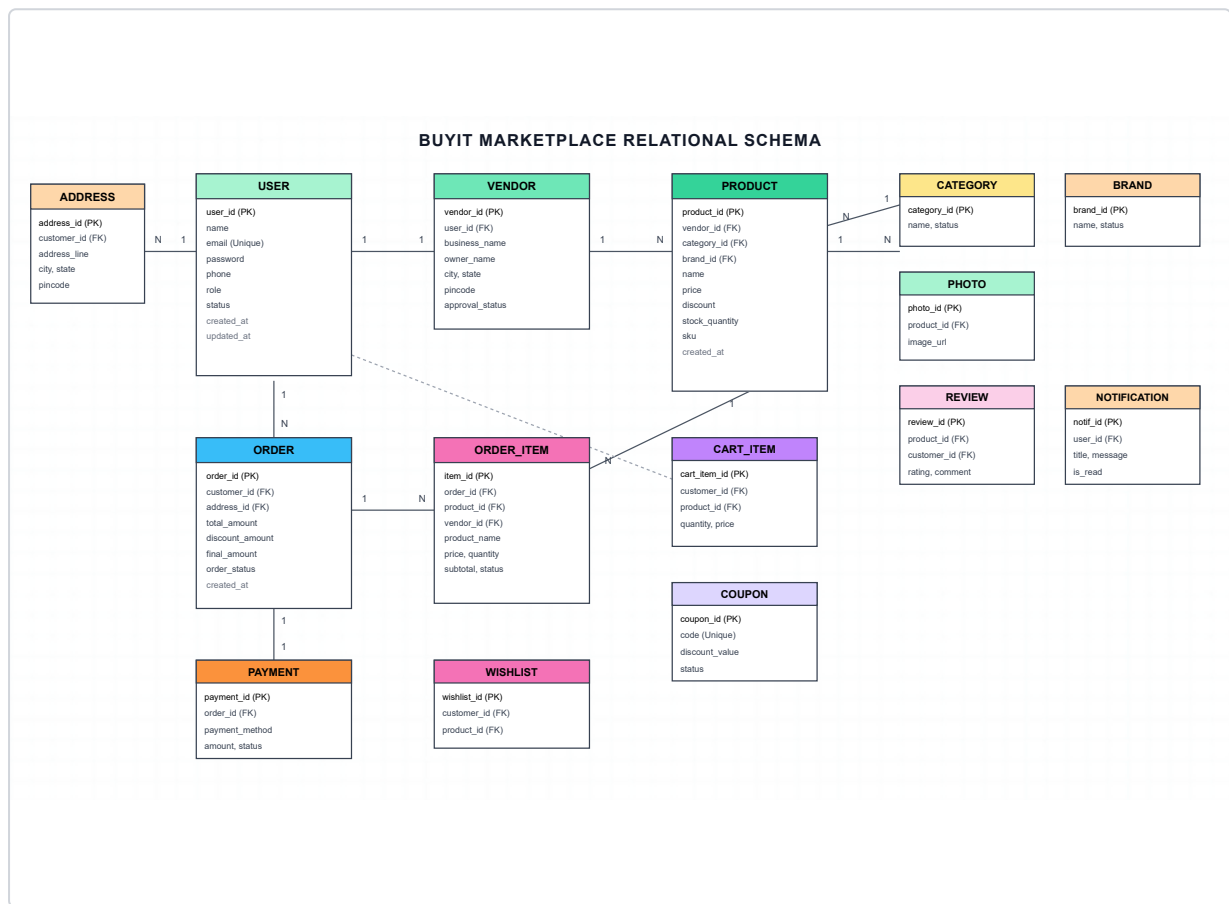
BUYIT MULTI-VENDOR E-COMMERCE PLATFORM

MySQL Database Implementation, JOIN Queries & Composite Key Testing

Database: MySQL 8.4

Report Type: Database Practical / Project Documentation

Prepared for: Academic Submission



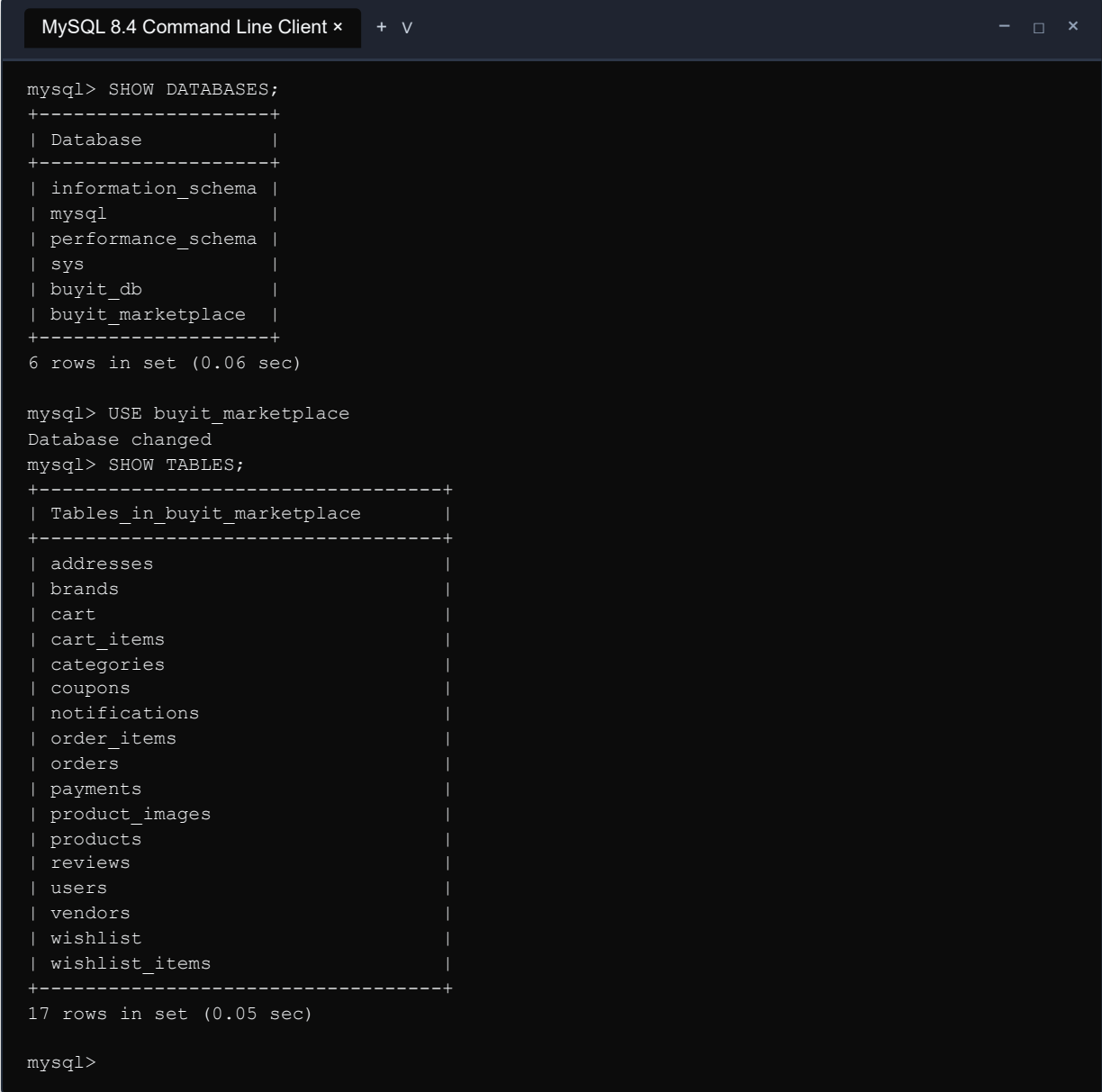
1. Objective

The objective of this practical work is to design and test the relational database for a Multi-Vendor E-Commerce Platform (BuyIt). The work covers table creation, primary and foreign keys, sample data insertion, JOIN operations, composite keys and intentional SQL error testing for debugging practice.

2. Database Structure

The database contains 17 tables: users, vendors, categories, brands, products, product_images, addresses, coupons, orders, order_items, payments, cart, cart_items, wishlist, wishlist_items, reviews, and notifications.

The relationships are implemented using primary keys and foreign keys. Examples include users -> vendors, vendors -> products, orders -> order_items, and so on.



```
MySQL 8.4 Command Line Client x + v

mysql> SHOW DATABASES;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sys                     |
| buyit_db                |
| buyit_marketplace       |
+-----+
6 rows in set (0.06 sec)

mysql> USE buyit_marketplace
Database changed
mysql> SHOW TABLES;
+-----+
| Tables_in_buyit_marketplace |
+-----+
| addresses                   |
| brands                      |
| cart                        |
| cart_items                  |
| categories                  |
| coupons                     |
| notifications                |
| order_items                 |
| orders                      |
| payments                    |
| product_images              |
| products                    |
| reviews                     |
| users                       |
| vendors                     |
| wishlist                    |
| wishlist_items              |
+-----+
17 rows in set (0.05 sec)

mysql>
```

3. Table Creation & Key Constraints

The schema was created using BIGINT primary keys with AUTO_INCREMENT where appropriate. Foreign keys were added to maintain referential integrity between related tables. UNIQUE constraints were used for fields like email.

The final schema contains 17 tables and the foreign key relationships were verified in MySQL 8.4 Command Line Client.

4. Sample Data Insertion

Sample records were inserted in foreign key-safe order: users, vendors, categories, brands, products, product_images, addresses, coupons, orders, order_items, payments, cart, cart_items, wishlist, wishlist_items, reviews, and notifications.

Expected Sample row counts after insertion:

```
MySQL 8.4 Command Line Client x

mysql> SELECT 'users' AS table_name, COUNT(*) AS records FROM users UNION ALL
-> SELECT 'vendors', COUNT(*) FROM vendors UNION ALL
-> SELECT 'categories', COUNT(*) FROM categories UNION ALL
-> SELECT 'brands', COUNT(*) FROM brands UNION ALL
-> SELECT 'products', COUNT(*) FROM products UNION ALL
-> SELECT 'product_images', COUNT(*) FROM product_images UNION ALL
-> SELECT 'addresses', COUNT(*) FROM addresses UNION ALL
-> SELECT 'orders', COUNT(*) FROM orders UNION ALL
-> SELECT 'order_items', COUNT(*) FROM order_items UNION ALL
-> SELECT 'payments', COUNT(*) FROM payments UNION ALL
-> SELECT 'cart', COUNT(*) FROM cart UNION ALL
-> SELECT 'cart_items', COUNT(*) FROM cart_items UNION ALL
-> SELECT 'wishlist', COUNT(*) FROM wishlist UNION ALL
-> SELECT 'wishlist_items', COUNT(*) FROM wishlist_items UNION ALL
-> SELECT 'reviews', COUNT(*) FROM reviews UNION ALL
-> SELECT 'coupons', COUNT(*) FROM coupons UNION ALL
-> SELECT 'notifications', COUNT(*) FROM notifications;
+-----+-----+
| table_name | records |
+-----+-----+
| users      | 5       |
| vendors    | 5       |
| categories | 6       |
| brands     | 11      |
| products   | 6       |
| product_images | 9       |
| addresses  | 6       |
| orders     | 8       |
| order_items | 4       |
| payments   | 3       |
| cart       | 6       |
| cart_items | 6       |
| wishlist   | 5       |
| wishlist_items | 9       |
| reviews    | 7       |
| coupons    | 5       |
| notifications | 5       |
+-----+-----+
17 rows in set (0.05 sec)

mysql>
```

5. JOIN OPERATIONS

Join operations were used to retrieve related information from multiple tables. The queries below use the actual table names instead of short aliases, making the relationship between tables easier to understand during the practical.

5.1 INNER JOIN

Returns only records that have matching values in both tables.

```
MySQL 8.4 Command Line Client x

mysql> SELECT
->     u.name AS customer_name,
->     p.name AS product_title
-> FROM users u
-> INNER JOIN orders o
->     ON u.id = o.customer_id
-> INNER JOIN order_items oi
->     ON o.id = oi.order_id
-> INNER JOIN products p
->     ON oi.product_id = p.id;
+-----+-----+
| customer_name | product_title |
+-----+-----+
| Customer One  | Apple iPhone 15 Pro 128GB Blue |
| Customer One  | Sony WH-1000XM5 Wireless Headphones |
| Rahul Sharma  | Nike Air Max 270 Men Running Shoes |
| Priya Patel   | Dell XPS 15 OLED Core i9 Laptop |
| Amit Kumar    | Fossil Gen 6 Smartwatch Smoke Brown |
| Sneha Reddy   | Prestige Iris 750W Mixer Grinder |
+-----+-----+
6 rows in set (0.00 sec)
```

5.2 LEFT JOIN

Returns all rows from the left table, plus matched rows from the right table. Unmatched right-side columns come back as NULL.

```
MySQL 8.4 Command Line Client x

mysql> SELECT
->     u.name AS customer_name,
->     o.final_amount
-> FROM users u
-> LEFT JOIN orders o
->     ON u.id = o.customer_id;
+-----+-----+
| customer_name | final_amount |
+-----+-----+
| Admin User    | NULL         |
| Rahul Sharma  | NULL         |
| Priya Patel   | 29990.00     |
| Amit Kumar    | 11495.00     |
| Sneha Reddy   | 189990.00    |
| Customer One  | 3499.00      |
+-----+-----+
6 rows in set (0.00 sec)
```

5.3 RIGHT JOIN

All rows from the right table, matched rows from the left, NULL where there is no match.

```
MySQL 8.4 Command Line Client x

mysql> SELECT
->     c.name AS category_name,
->     p.name AS product_title
-> FROM products p
-> RIGHT JOIN categories c
->     ON p.category_id = c.id;
+-----+-----+
| category_name | product_title |
+-----+-----+
| Electronics  | Apple iPhone 15 Pro 128GB Blue |
| Electronics  | Sony WH-1000XM5 Wireless Headphones |
| Shoes        | Nike Air Max 270 Men Running Shoes |
| Laptops      | Dell XPS 15 OLED Core i9 Laptop |
| Watches      | Fossil Gen 6 Smartwatch Smoke Brown |
| Home & Kitchen | Prestige Iris 750W Mixer Grinder |
+-----+-----+
6 rows in set (0.00 sec)
```

5.4 FULL JOIN

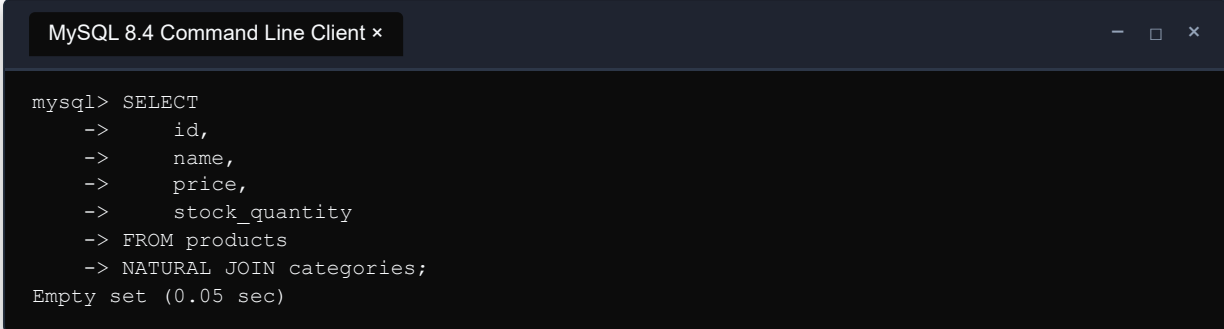
All rows from both right and left tables, with NULL on whichever side lacks a match.

```
MySQL 8.4 Command Line Client x

mysql> SELECT
->     u.name AS user_name,
->     v.business_name
-> FROM users u
-> LEFT JOIN vendors v
->     ON u.id = v.user_id
-> UNION
-> SELECT
->     u.name AS user_name,
->     v.business_name
-> FROM users u
-> RIGHT JOIN vendors v
->     ON u.id = v.user_id;
+-----+-----+
| user_name | business_name |
+-----+-----+
| Admin User | NULL          |
| Rahul Sharma | Sharma Electronics Hub |
| Priya Patel | Patel Fashion World |
| Amit Kumar | Kumar Home Appliances |
| Sneha Reddy | Reddy Sports Gear |
| Customer One | NULL          |
+-----+-----+
6 rows in set (0.03 sec)
```

5.5 NATURAL JOIN

Automatically joins two tables on all columns that share the same name—you don't specify the ON condition yourself. MySQL compares every identically named column and matches rows where all of them are equal.

A screenshot of the MySQL 8.4 Command Line Client window. The window has a title bar with the text "MySQL 8.4 Command Line Client" and standard window control buttons (minimize, maximize, close). The main area shows a SQL query being executed. The prompt "mysql>" is followed by the query: "SELECT id, name, price, stock_quantity FROM products NATURAL JOIN categories;". Below the query, the output is "Empty set (0.05 sec)".

```
mysql> SELECT
->     id,
->     name,
->     price,
->     stock_quantity
-> FROM products
-> NATURAL JOIN categories;
Empty set (0.05 sec)
```